Qwen2.5-VL-3B-Instruct — end-to-end architecture

3.75 B parameters = vision encoder 632.0 M + patch merger 36.7 M + language model 3.09 B (token embedding shared with the LM head)

Shapes [rows, features]: P = image patches of 14×14 px, N = P / 4 visual tokens,
L = prompt length in tokens (text + N), d = 2048 (LLM width), V = 151,936 (vocab rows)
Example used in all figures: 448×336 px image + “What is in the image?”
→ P = 32×24 = 768, N = 16×12 = 192, L = 219

VISION PATH — runs once per image

Image
any size H × W
RGB

Image processor (CPU)
resize: H, W → multiples of 28 px
(aspect ratio kept, pixel budget
min_pixels … max_pixels)
normalize (CLIP mean / std)
repeat the frame ×2 (temporal patch)
cut into 14×14 px patches

[P,1176]

Patch embedding
Conv3D 3→1280
kernel = stride
= (2, 14, 14)
≡ Linear 1176→1280
no bias

[P,1280]

Vision Transformer ×32
width 1280 · 16 heads × 80
2D RoPE on q, k (row, col)
window attention: 8×8 patches
full attention: blocks 7, 15, 23, 31
SwiGLU MLP 1280→3420→1280
bidirectional (no causal mask)

[P,1280]

Patch merger (projector)
the 4 patches of a 2×2 square:
concatenated (all kept), mixed
RMSNorm, concat 4×1280 = 5120
Linear 5120→5120 · GELU
Linear 5120→2048

[N, 2048]

TEXT PATH

Prompt
question
+ image

Chat template
<|im_start|>system … <|im_end|>
<|im_start|>user
<|vision_start|><|image_pad|><|vision_end|>
What is in the image?<|im_end|>
<|im_start|>assistant
<|image_pad|> is repeated N times

text

Tokenizer
byte-level BPE
151,643 tokens
+ 22 special
= 151,665 ids

[L] ids

Token embedding
lookup table V × d
= 151,936 × 2048
rows ≥ 151,665: unused padding
(text tokens → vectors)

[L,2048]

Insert image features
the N rows at <|image_pad|>
positions are replaced
by the N merger outputs

[L,2048]

LANGUAGE MODEL

3D position ids (M-RoPE)
text: t = h = w = position
image: t shared, h = row, w = col
from token ids + image grid
→ cos / sin used in every layer

[3, L]

Decoder layer ×36
pre-RMSNorm + residual
self-attention (GQA, causal)
16 query heads, 2 KV heads
head dim 128, M-RoPE on q, k
SwiGLU MLP 2048→11008→2048
KV cache for decoding

[L, 2048]

Final RMSNorm

[1, 2048] last pos.

LM head
Linear 2048 → 151,936
no bias
weights tied to token embedding

[1, 151,936] logits

Next token
softmax → greedy / sampling
only the last position
is needed

decode loop: the new token is embedded and fed back,
one token per step; earlier keys / values come from the KV cache

Sizes of this model (from config.json)

```
Vision encoder (same shape in 3B and 7B)
  width 1280, 32 blocks, 16 heads × 80
  MLP 1280→3420→1280, window 112 px
Language model
  width d = 2048, 36 layers
  16 q-heads / 2 KV-heads × 128
  MLP 2048→11008→2048
  vocab rows 151,936, tied head: yes
  RoPE θ = 1,000,000, RMSNorm ε = 1e-06
```

Where the parameters are



ViT (patch embed + 32 blocks): 632.0 M (16.8 %)
patch merger: 36.7 M (1.0 %)
token embedding = LM head: 311.2 M (8.3 %)
36 decoder layers + final norm: 2.77 B (73.9 %)

bf16 weights: 7.5 GB · KV cache: 36 KiB per token · decoder ≈ 6.2 GFLOPs per generated token (linear layers)

Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

Qwen2.5-VL-7B-Instruct — end-to-end architecture

8.29 B parameters = vision encoder 632.0 M + patch merger 44.6 M + language model 7.62 B

Shapes [rows, features]: P = image patches of 14×14 px, N = P / 4 visual tokens,
L = prompt length in tokens (text + N), d = 3584 (LLM width), V = 152,064 (vocab rows)
Example used in all figures: 448×336 px image + "What is in the image?"
→ P = 32×24 = 768, N = 16×12 = 192, L = 219

VISION PATH — runs once per image

Image
any size H × W
RGB

→

Image processor (CPU)
resize: H, W → multiples of 28 px
(aspect ratio kept, pixel budget
min_pixels … max_pixels)
normalize (CLIP mean / std)
repeat the frame ×2 (temporal patch)
cut into 14×14 px patches

[P,1176]

Patch embedding
Conv3D 3→1280
kernel = stride
= (2, 14, 14)
≡ Linear 1176→1280
no bias

[P,1280]

Vision Transformer ×32
width 1280 · 16 heads × 80
2D RoPE on q, k (row, col)
window attention: 8×8 patches
full attention: blocks 7, 15, 23, 31
SwiGLU MLP 1280→3420→1280
bidirectional (no causal mask)

[P,1280]

Patch merger (projector)
the 4 patches of a 2×2 square:
concatenated (all kept), mixed
RMSNorm, concat 4×1280 = 5120
Linear 5120→5120 · GELU
Linear 5120→3584

[N, 3584]

TEXT PATH

Prompt
question
+ image

→

Chat template
<|im_start|>system … <|im_end|>
<|im_start|>user
<|vision_start|><|image_pad|><|vision_end|>
What is in the image?<|im_end|>
<|im_start|>assistant
<|image_pad|> is repeated N times

text

Tokenizer
byte-level BPE
151,643 tokens
+ 22 special
= 151,665 ids

[L] ids

Token embedding
lookup table V × d
= 152,064 × 3584
rows ≥ 151,665: unused padding
(text tokens → vectors)

[L,3584]

Insert image features
the N rows at <|image_pad|>
positions are replaced
by the N merger outputs

[L,3584]

LANGUAGE MODEL

3D position ids (M-RoPE)
text: t = h = w = position
image: t shared, h = row, w = col
from token ids + image grid
→ cos / sin used in every layer

[3, L]

Decoder layer ×28
pre-RMSNorm + residual
self-attention (GQA, causal)
28 query heads, 4 KV heads
head dim 128, M-RoPE on q, k
SwiGLU MLP 3584→18944→3584
KV cache for decoding

[L, 3584]

Final RMSNorm

[1, 3584] last pos.

LM head
Linear 3584 → 152,064
no bias
own weights (not tied)

[1, 152,064] logits

Next token
softmax → greedy / sampling
only the last position
is needed

decode loop: the new token is embedded and fed back,
one token per step; earlier keys / values come from the KV cache

Sizes of this model (from config.json)

```
Vision encoder (same shape in 3B and 7B)
  width 1280, 32 blocks, 16 heads × 80
  MLP 1280→3420→1280, window 112 px
Language model
  width d = 3584, 28 layers
  28 q-heads / 4 KV-heads × 128
  MLP 3584→18944→3584
  vocab rows 152,064, tied head: no
  RoPE θ = 1,000,000, RMSNorm ε = 1e-06
```

Where the parameters are



- ViT (patch embed + 32 blocks): 632.0 M (7.6 %)
- patch merger: 44.6 M (0.5 %)
- token embedding: 545.0 M (6.6 %)
- 28 decoder layers + final norm: 6.53 B (78.7 %)
- LM head: 545.0 M (6.6 %)

bf16 weights: 16.6 GB · KV cache: 56 KiB per token · decoder ≈ 14.1 GFLOPs per generated token (linear layers)

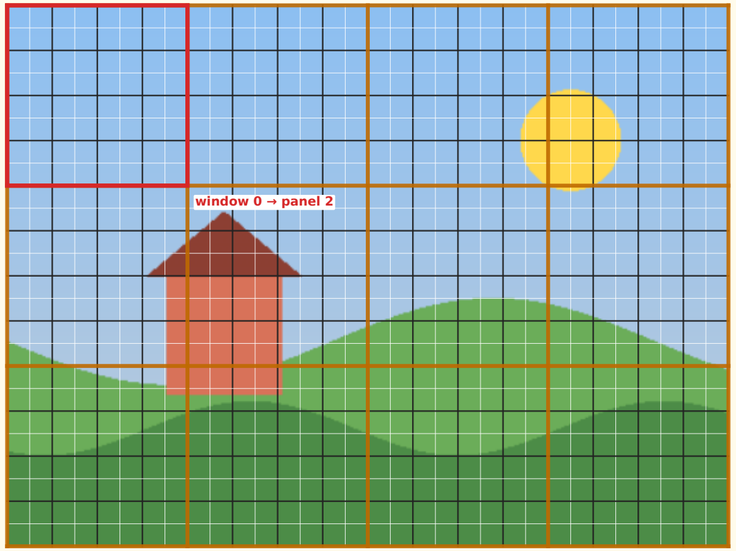
Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

From pixels to visual tokens

What the image processor and the ViT do with a picture (identical in 3B and 7B). Example: a 450×340 photo.

1 · Resize, cut into patches, group, window

450×340 → 448×336 px (nearest multiples of 28)



thin: 14×14 px patch

32 × 24 = 768 patches (P)

medium: 28×28 px = 2×2 patches

= 1 LLM token: 16 × 12 = 192 (N)

thick: 112×112 px = 8×8 patches

= 1 attention window: 4 × 3 = 12

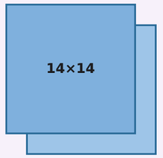
Windows at the right / bottom edge can be smaller.

2 · Sequence order inside one window

0	1	4	5	8	9	12	13
2	3	6	7	10	11	14	15
16	17	20	21	24	25	28	29
18	19	22	23	26	27	30	31
32	33	36	37	40	41	44	45
34	35	38	39	42	43	46	47
48	49	52	53	56	57	60	61
50	51	54	55	58	59	62	63

- numbers = position in the ViT sequence (window 0)
- the 4 patches of a 2×2 unit (green outline) are consecutive — the merger later joins rows 4k … 4k+3
- units go row by row inside a window; windows go row by row over the image (window 1 starts at 64)
- window attention: a patch attends to the ≤ 64 patches of its own window (28 of 32 blocks)
- full attention (blocks 7, 15, 23, 31): to all 768
- after the merger, the 192 tokens are put back in plain row-major order (12 rows × 16 columns)

3 · One patch → one vector → one token



14 × 14 px, RGB, the image copied into 2 frames (videos: 2 real frames)

3 · 2 · 14 · 14 = 1176 numbers

Patch embedding: Linear 1176 → 1280

patch vector, 1280 numbers

… 32 vision blocks (shape stays [P, 1280]) …

patch 1 | **patch 2** | **patch 3** | **patch 4**

RMSNorm each, concat → 5120 numbers (all 4 kept)

Merger MLP: 5120 → 5120 → d

1 visual token, d numbers

d = 2048 (3B) · d = 3584 (7B)

It replaces one <|image_pad|> embedding, so the LLM reads it like any other token.

One token covers 28 × 28 px of the image.

Merger step by step: next page.

Visual tokens N = (H / 28) × (W / 28) e.g. (336 / 28) × (448 / 28) = 12 × 16 = 192 Patches P = 4 N = 768
Default pixel budget: min_pixels 3,136 … max_pixels 12,845,056 → 4 … 16,384 tokens per image. This harness: 256 … 2,048 tokens (jetson/models/qwen2_5_vl/model.sh).

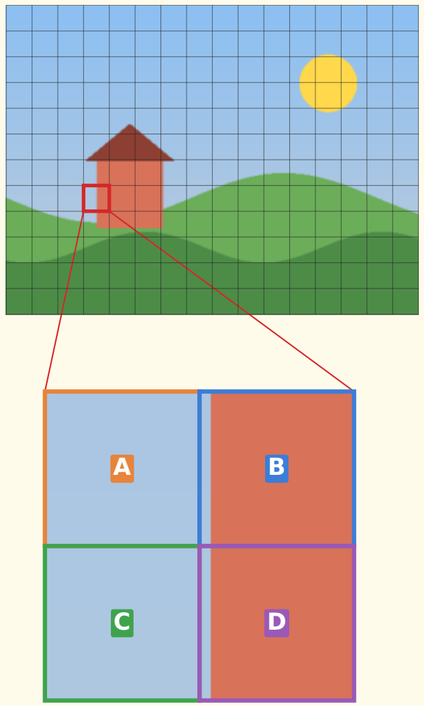
Image processor: Qwen2VLImageProcessor (smart_resize, patch flattening). Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

Patch merger: 4 patches → 1 token, nothing is dropped

Runs after the 32 vision blocks. It concatenates the 4 patch vectors of every 2×2 square, mixes them with a 2-layer MLP and projects to the LLM width. Bar heights are to scale (1 unit = 128 numbers).

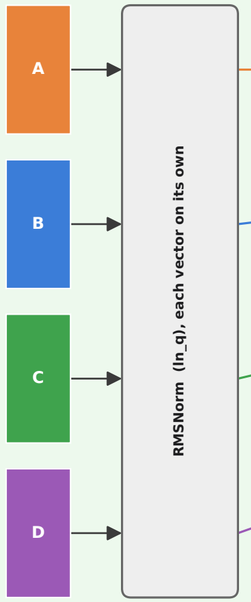
Every 2×2 square of the image is processed this way, independently (768 patches → 192 tokens in the example).

1 · One 2×2 square of the image



28 × 28 px = 4 patches of 14 × 14

2 · Four separate vectors



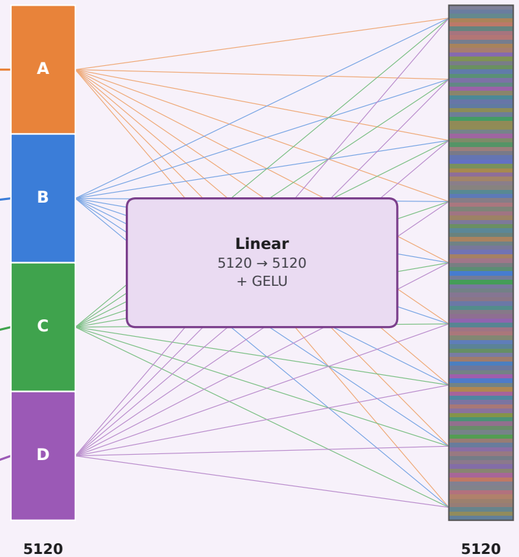
one 1280-d vector per patch,

output of the vision blocks

(already mixed with its context

by attention)

3 · Concatenate, then mix

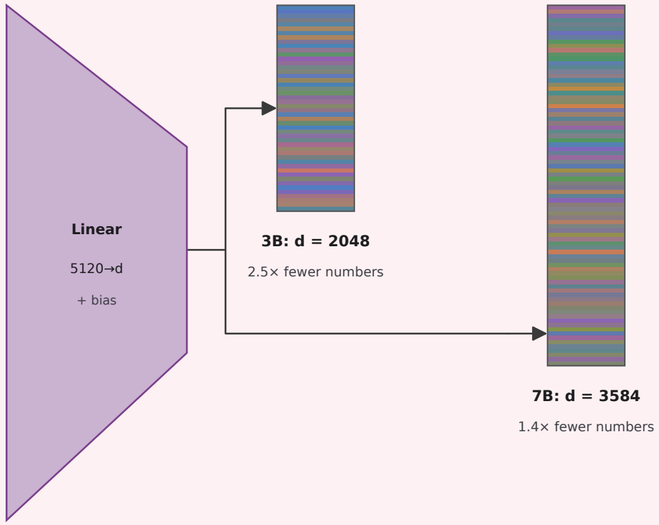


concat = a reshape: A|B|C|D stacked, all 5120 kept

Linear: every output number is a weighted

sum of all 5120 inputs → A, B, C, D combined

4 · Project to the LLM width → 1 token



one or the other, depending on the model

1 visual token = the vector that replaces

one <|image_pad|> in the LLM input

What the merger is NOT

✗ **keep 1 patch, drop 3**

no selection: no patch is discarded

✗ **average the 4 patches**

no fixed pooling: the weights are learned

✓ **learned mix of all 4**

concat (lossless) + MLP; only the final 5120 → d step compresses

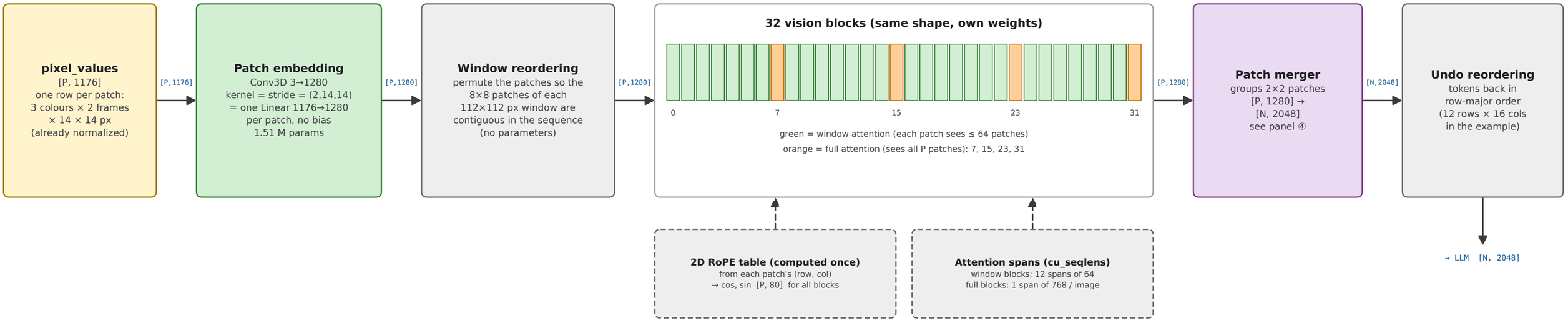
Why it is there

- 4× shorter image sequence: 768 patches → 192 LLM tokens, so 4× less KV cache and ~16× less attention work between image tokens in every decoder layer.
- It is the only bridge between the two models: it maps vision features into the LLM's embedding space (width 2048 in 3B, 3584 in 7B).
- Small: 36.7 M (3B) / 44.6 M (7B) parameters, ~1 % of the model; stored as visual.merger.*
- Runs in window order; the ViT puts the tokens back in row-major order afterwards.

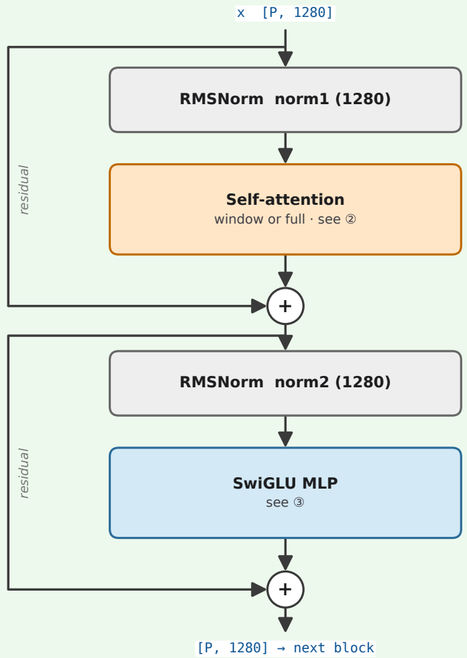
Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

Qwen2.5-VL-3B-Instruct — vision encoder in detail

ViT 632.0 M + patch merger 36.7 M. Identical layout in 3B and 7B except the merger's last Linear (5120→2048). Example numbers: P = 768 patches, N = 192 tokens.

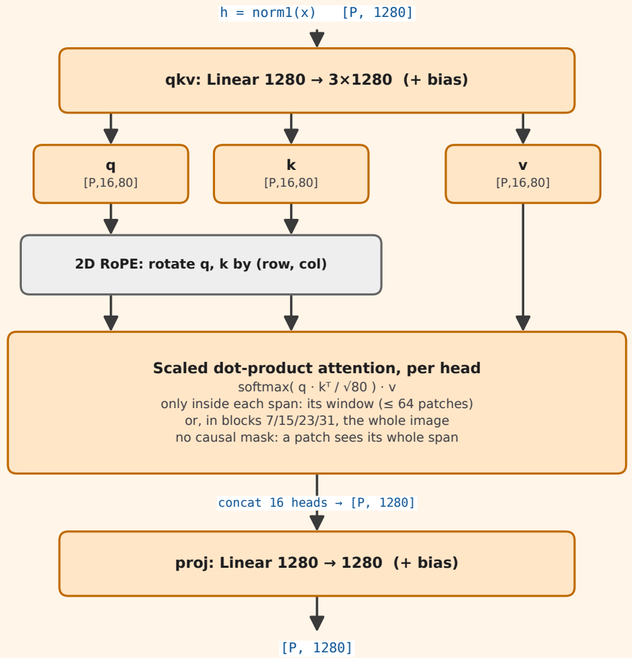


① Vision block (×32)



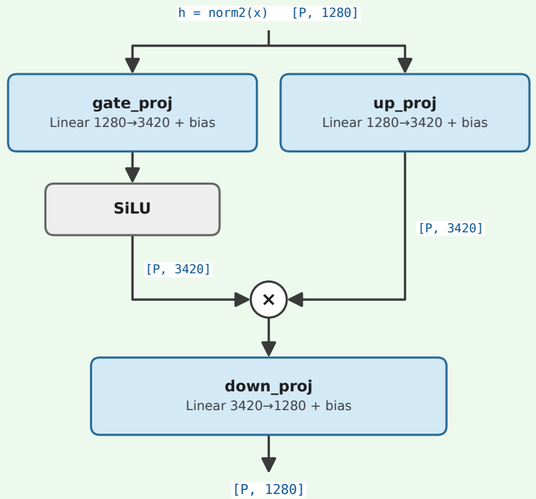
19.70 M params / block (×32 = 630.5 M)

② Vision self-attention



6.56 M params / block

③ Vision MLP (SwiGLU)



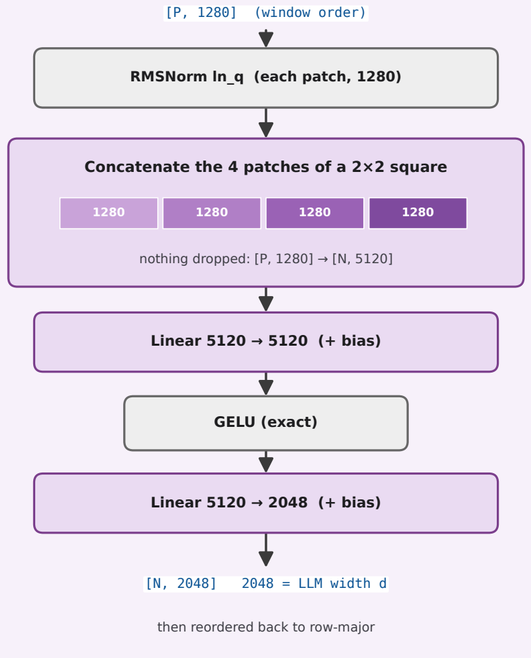
MLP(h) = down(SiLU(gate(h)) ⊙ up(h))

⊙ = element-wise product: the SiLU branch

decides how much of each of the 3420 features passes

13.14 M params / block

④ Patch merger



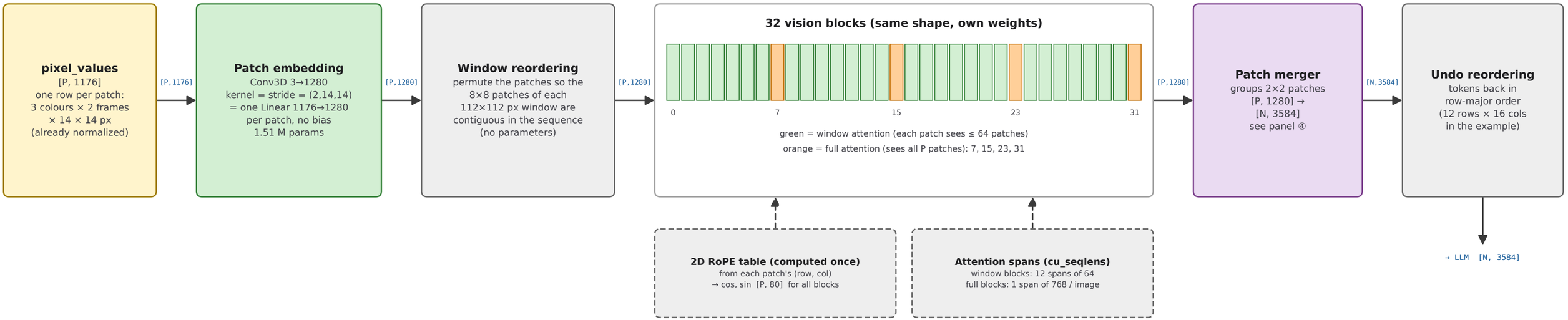
then reordered back to row-major

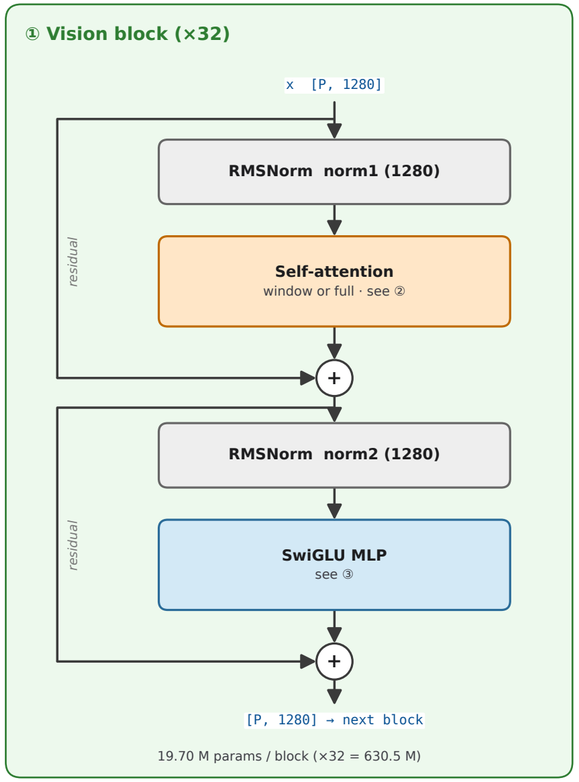
36.71 M params

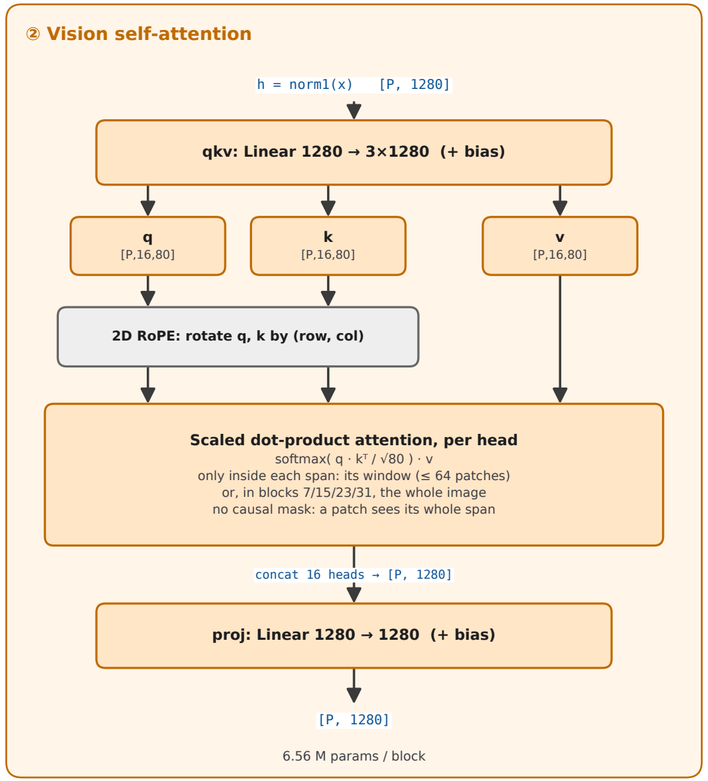
Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

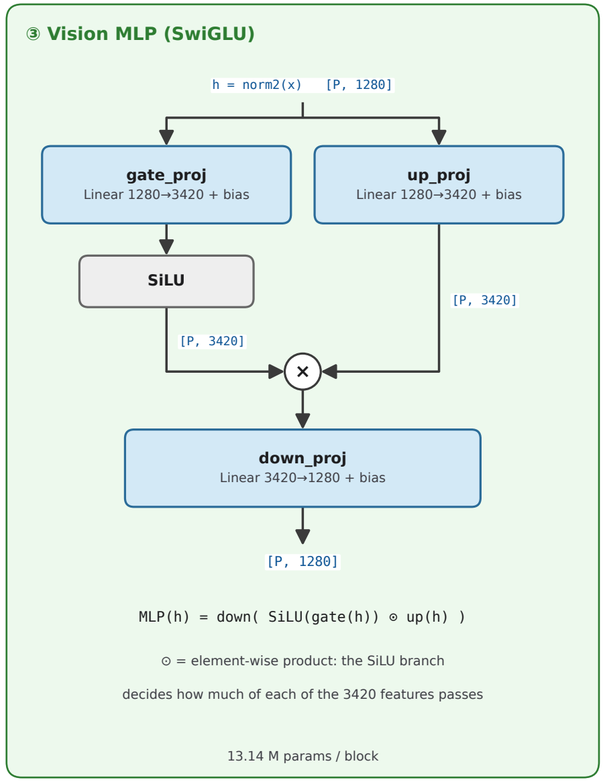
Qwen2.5-VL-7B-Instruct — vision encoder in detail

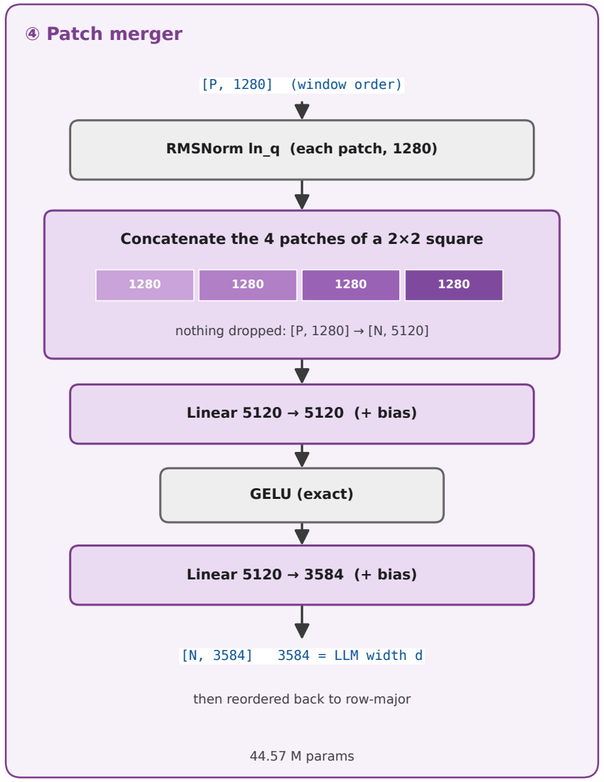
ViT 632.0 M + patch merger 44.6 M. Identical layout in 3B and 7B except the merger's last Linear (5120→3584). Example numbers: P = 768 patches, N = 192 tokens.









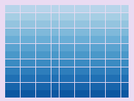


Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

Where is each token? Rotary position encodings

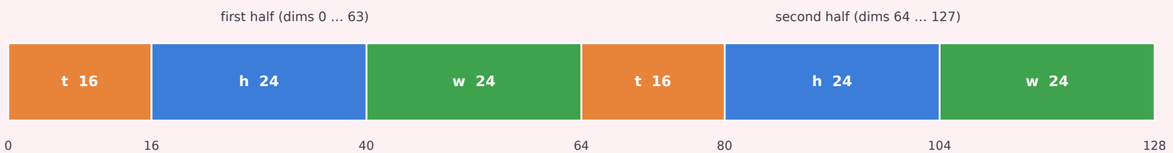
LLM: 3D multimodal RoPE (M-RoPE, head dim 128). ViT: 2D RoPE (head dim 80). Same in 3B and 7B. No learned position embeddings anywhere.

A · Position ids of the example prompt (L = 219 tokens; image = 12 × 16 tokens) — values from Qwen2_5_VLModel.get_rope_index

tokens	<|im_start|>system You are a helpful assistant.<|im_end|> <|im_start|>user	<|vision_ start|>	 192 × <|image_pad|> 12 rows (h) × 16 columns (w) row-major, as the merger outputs them	<|vision_ end|>	What is in the image?<|im_end|> <|im_start|> assistant	1st new token
sequence index	0 … 13	14	15 … 206	207	208 … 218	219
t (time)	**0 … 13**	**14**	**15 (all 192)**	**31**	**32 … 42**	**43**
h (height)	**0 … 13**	**14**	**15 + row = 15 … 26**	**31**	**32 … 42**	**43**
w (width)	**0 … 13**	**14**	**15 + col = 15 … 30**	**31**	**32 … 42**	**43**

- Text tokens get t = h = w = their running position, so for pure text M-RoPE is exactly the 1D RoPE of Qwen2.5.
- All image tokens share t (one frame) and take h, w from their row / column in the 12 × 16 grid, offset by the start position 15.
- Text after the image resumes at max + 1 = 31: the 192 image tokens advance the position by only 16 (max(rows, cols)), not by 192.
- Decoding: every new token gets t = h = w = sequence index − 176 (the stored rope delta = 43 − 219), e.g. index 219 → 43, 220 → 44.
- Videos: t = frame-group index × seconds per group × 2 (tokens_per_second), so t follows real time.

B · LLM: which of the 128 dims of a head see t, h or w



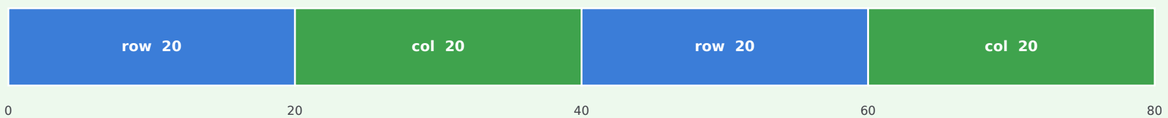
RoPE rotates dim i together with dim i + 64, at frequency θ^(−2i/128), θ = 10⁶.

Of the 64 rotation pairs, the 16 fastest use t, the next 24 use h,

the slowest 24 use w   (rope_scaling.mrope_section = [16, 24, 24]).

Applied to q and k in every layer; v is not rotated.

C · ViT: 2D RoPE inside the 80 dims of a vision head



Dim j rotates with dim j + 40. Row and column each use the same 20

frequencies 10000^(−2k/40), k = 0 … 19 (θ = 10⁴).

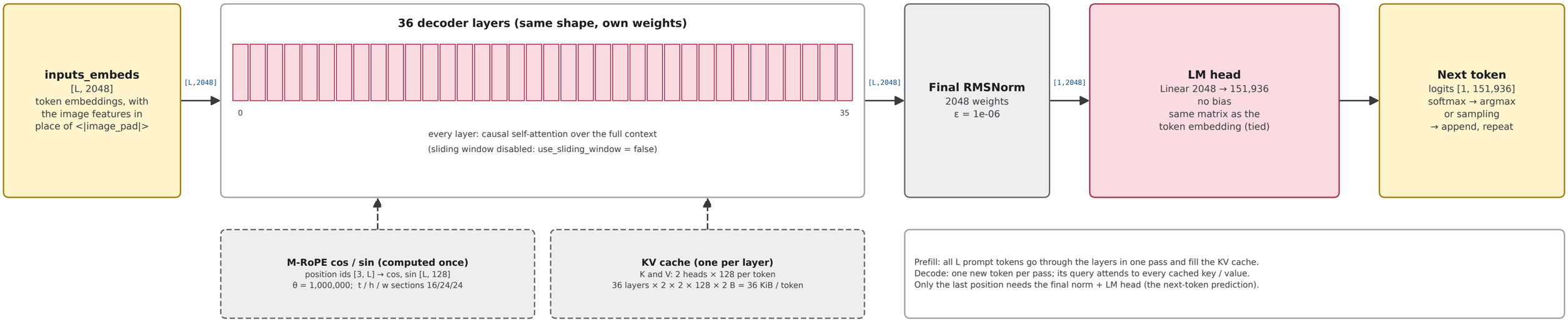
Positions = the patch's (row, col) in the 14-px grid of its own image.

cos / sin are computed once and reused by all 32 blocks.

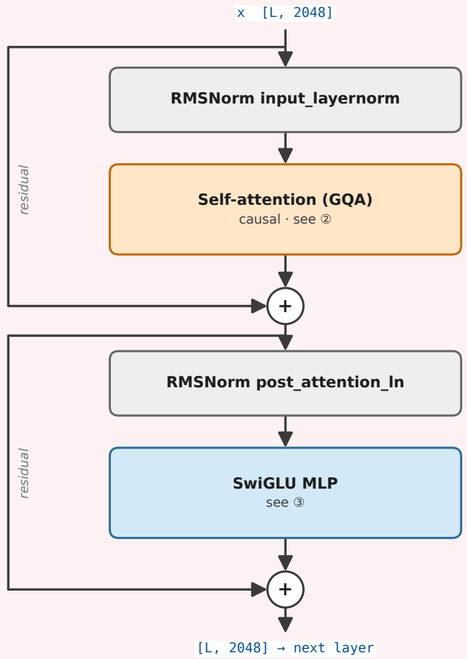
Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

Qwen2.5-VL-3B-Instruct — language model in detail

Qwen2.5 decoder: 3.09 B parameters, width 2048, 36 layers, 16 query heads / 2 KV heads × 128, vocab rows 151,936, embedding tied with the LM head

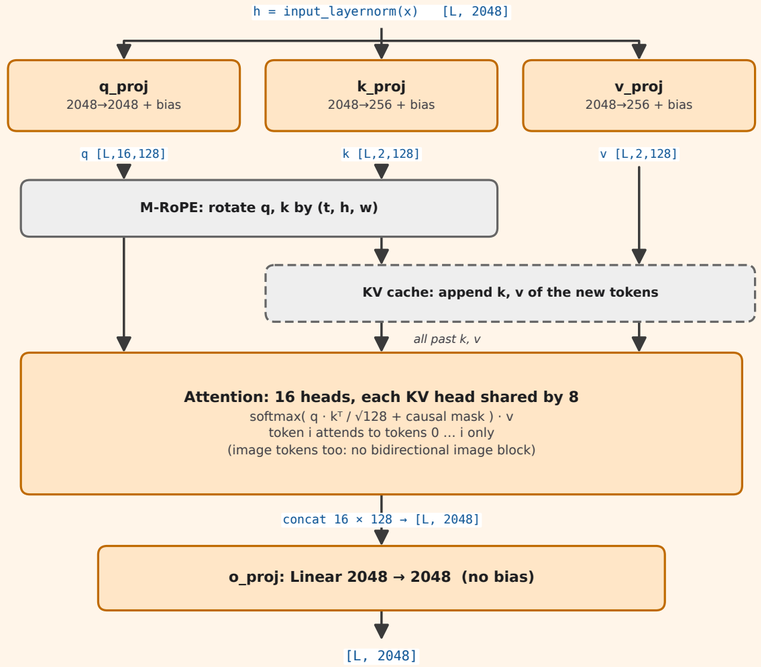


① Decoder layer (×36)



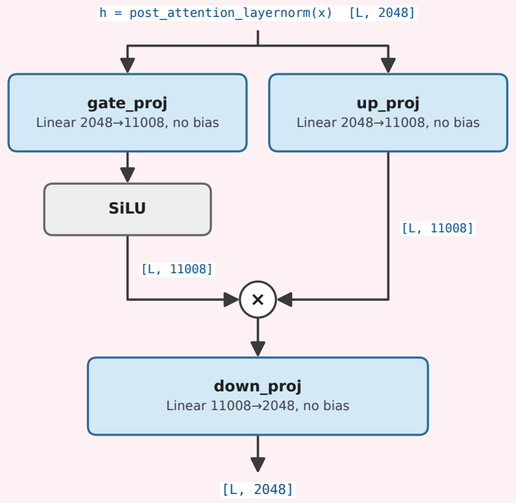
77.08 M params / layer (×36 = 2.77 B)

② Grouped-query self-attention



9.44 M params / layer

③ MLP (SwiGLU)

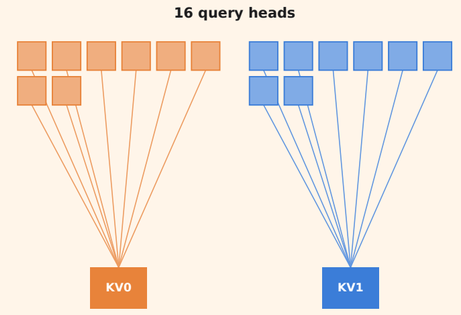


MLP(h) = down(SiLU(gate(h)) ⊙ up(h))

⊙ = element-wise product: the SiLU branch decides how much of each of the 11008 features passes

67.63 M params / layer

④ Who shares what (GQA)



2 key/value heads

Each group of 8 query heads reads the same key and value head.

K / V cache per token and layer:

2 heads × 128 × 2 (K, V) × 2 bytes

= 1,024 B → × 36 layers

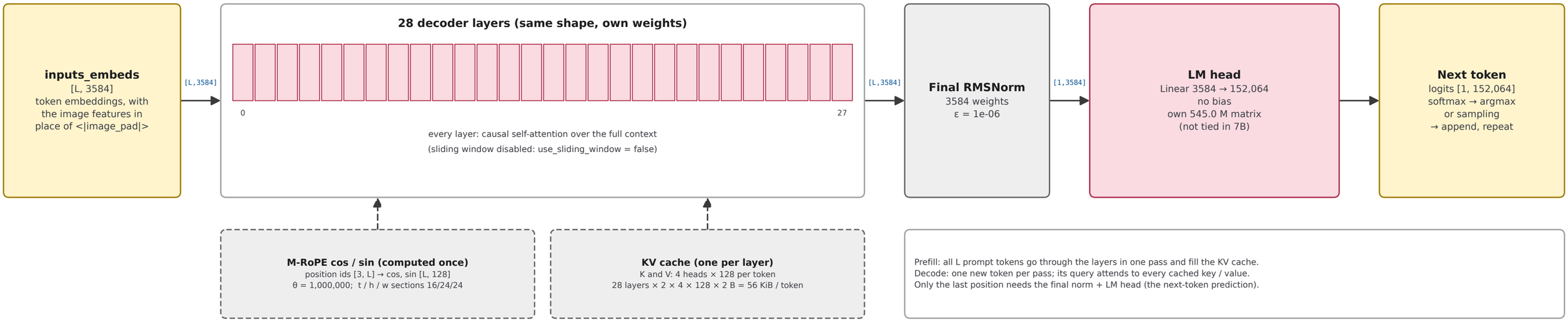
= 36 KiB per token

(8× smaller than with 16 KV heads)

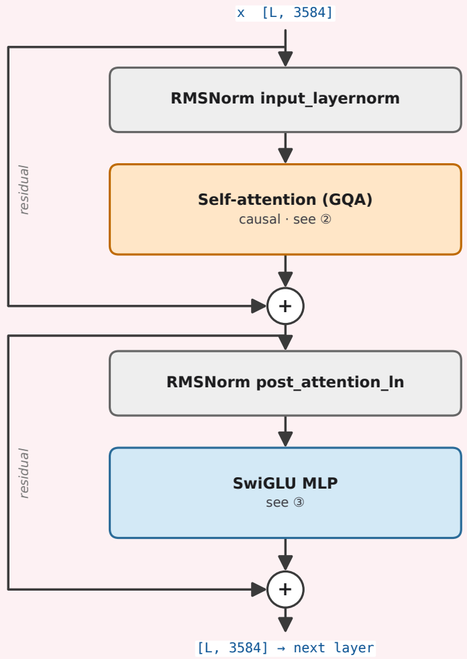
Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

Qwen2.5-VL-7B-Instruct — language model in detail

Qwen2.5 decoder: 7.62 B parameters, width 3584, 28 layers, 28 query heads / 4 KV heads × 128, vocab rows 152,064, separate LM head

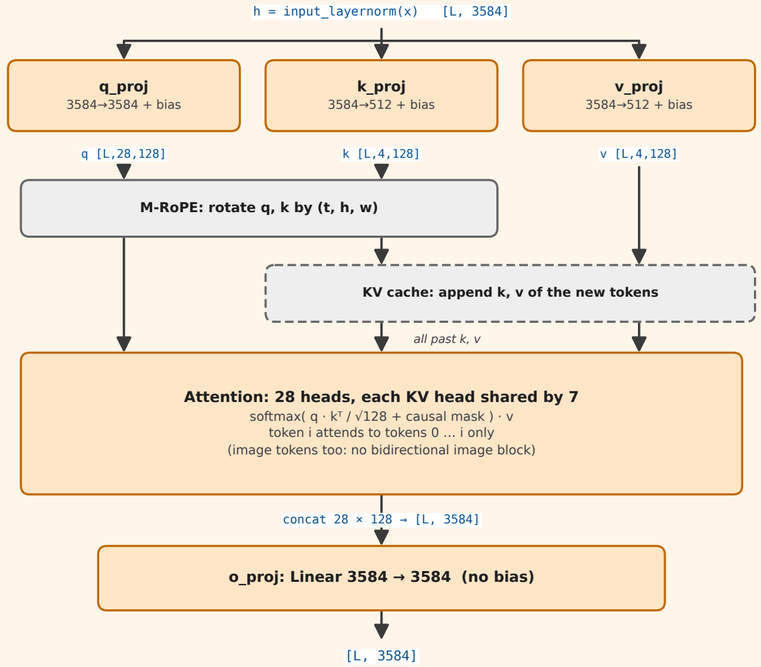


① Decoder layer (×28)



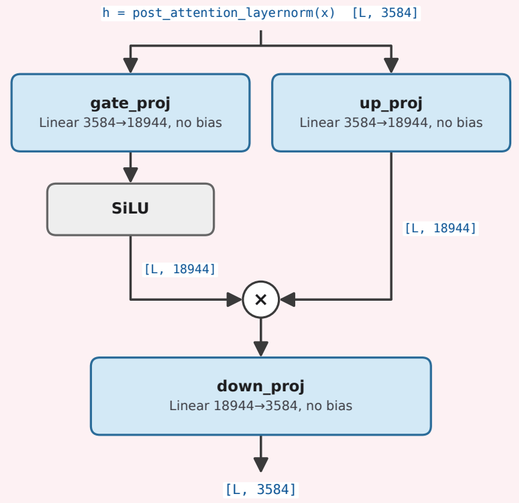
233.06 M params / layer (×28 = 6.53 B)

② Grouped-query self-attention



29.36 M params / layer

③ MLP (SwiGLU)

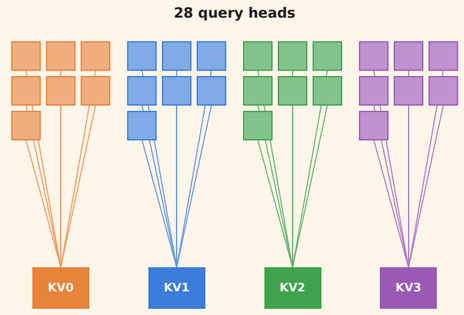


`MLP(h) = down( SiLU(gate(h)) ⊙ up(h) )`

⊙ = element-wise product: the SiLU branch decides how much of each of the 18944 features passes

203.69 M params / layer

④ Who shares what (GQA)



Each group of 7 query heads reads the same key and value head.

K / V cache per token and layer:

4 heads × 128 × 2 (K, V) × 2 bytes

= 2,048 B → × 28 layers

= 56 KiB per token

(7× smaller than with 28 KV heads)

Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py

Qwen2.5-VL-3B vs 7B — what changes

Same recipe; the vision encoder keeps its shape, the language model is wider (7B) or deeper (3B). Highlighted rows differ.

	3B	7B	note
Total parameters	**3.75 B**	**8.29 B**	
bf16 weights	**7.5 GB**	**16.6 GB**	2 bytes per parameter
VISION ENCODER			
width · blocks · heads × dim	1280 · 32 · 16×80	1280 · 32 · 16×80	same shape, separately trained weights
MLP (SwiGLU, with bias)	1280→3420→1280	1280→3420→1280	
full-attention blocks	7, 15, 23, 31	7, 15, 23, 31	the others use 112×112 px windows
ViT parameters	632.0 M	632.0 M	
patch merger	**5120→5120→2048**	**5120→5120→3584**	output = LLM width
merger parameters	**36.7 M**	**44.6 M**	
LANGUAGE MODEL			
hidden size d	**2048**	**3584**	
decoder layers	**36**	**28**	3B is deeper but much narrower
query heads / KV heads	**16 / 2**	**28 / 4**	query heads per KV head: 8 vs 7
head dim	128	128	
MLP intermediate size	**11,008**	**18,944**	5.38 × d vs 5.29 × d
vocab rows (embedding / LM head)	**151,936**	**152,064**	151,665 real tokens, the rest padding
LM head	**tied to embedding**	**separate (+545.0 M)**	
RoPE θ · M-RoPE sections	1,000,000 · [16, 24, 24]	1,000,000 · [16, 24, 24]	
language-model parameters	**3.09 B**	**7.62 B**	incl. embedding (and LM head)
KV cache per token (bf16)	**36 KiB**	**56 KiB**	layers × 2 × KV heads × 128 × 2 B
decoder GFLOPs per token	**6.2**	**14.1**	2 × (layer + LM head weights), w/o attention

Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2_5_vl.py · generated by make_diagrams.py